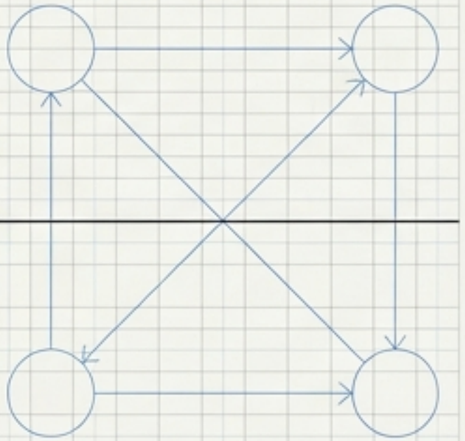
DOC_ID: MGRAPH_UI_PROTO_V1
AUTH: SYSTEM_ARCHITECT
DATE: 2026-02-02

ISSUES UI:
FRONTEND DEVELOPMENT BRIEFING

Operational Playbook for LLM Sessions & Developers



ROLE
Frontend-Only Development

SYSTEM
Graph-Based Issue Tracking

VERSION
1.0 (Active)

OPERATIONAL SCOPE AND BOUNDARIES

THE GREEN ZONE (AUTHORIZED)

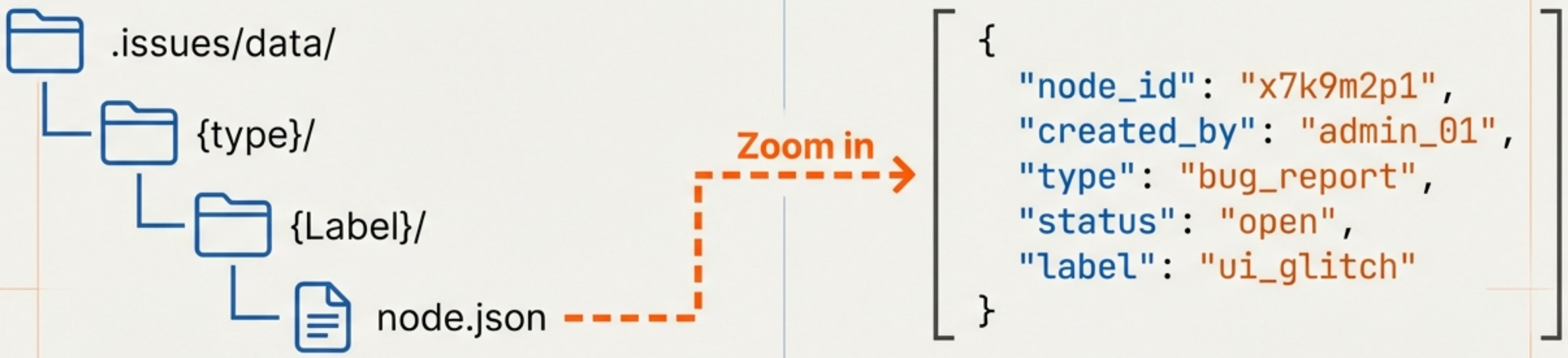
- Modify files in: `mgraph_ai_ui_html_transformation_workbench__ui/issues_ui/`
- Create/Update briefs in: `docs/dev-briefs/issues-ui/`
- Modify data in: `.issues/data/`

THE RED ZONE (RESTRICTED)

- **NO** modifications to backend Python: `mgraph_ai_ui_html_transformation_workbench/`
- **NO** modifications to API endpoints.
- **NO** changes to server-side logic.

PROTOCOL NOTICE: Requirement for backend changes? Initiate Briefing Protocol (See Slide 9).

THE DATA STRUCTURE: NODES & GRAPHS



CRITICAL SYNTAX RULE

When creating new issues, the '**node_id**' and '**created_by**' fields MUST be exactly 8 alphanumeric characters.

valid: `'x7k9m2p1'` ✅ invalid: `'new-bug'` ❌

ISSUE TAXONOMY & LIFECYCLE LOGIC

BUG	backlog	confirmed	in-progress	testing	resolved	closed
TASK	backlog	todo	in-progress	review	done	
FEATURE	proposed	approved	in-progress	released		
VERSION	planned	in-progress	released	deprecated		
USER-STORY	draft	ready	in-progress	implemented	validated	

PROTOCOL NOTICE: Lifecycle transitions are managed automatically. Manual overrides require explicit authorization. (See Slide 11).

METHODOLOGY: ITERATIVE FLOW DEVELOPMENT (IFD)

The Principle of Surgical Overrides

CONSOLIDATION LAYER (v0.2.0)

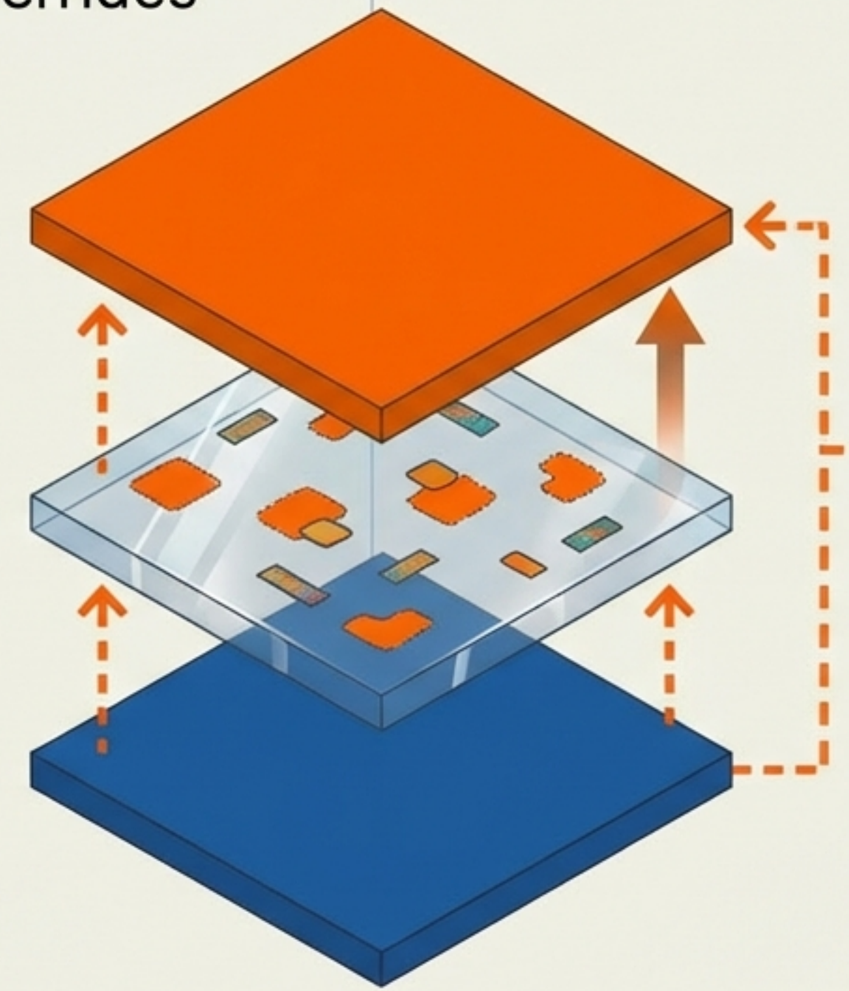
Merged Base. Consolidates all v0.1.x patches into new self-contained core.

PATCH LAYER (v0.1.1 - v0.1.N)

Surgical overrides only. Patches specific functions without file replacement.

BASE LAYER (v0.1.0)

Complete base files.
Self-contained system.



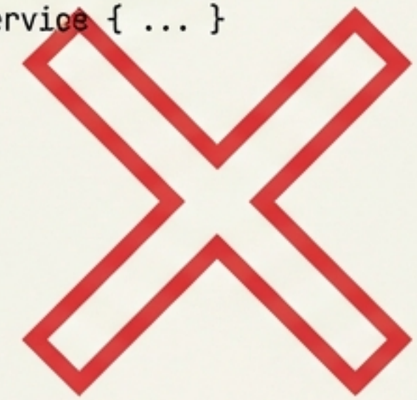
⚠ WE DO NOT COPY FILES.
WE PATCH THEM.

THE SURGICAL OVERRIDE PATTERN

Required Coding Standard: IIFE

DESTRUCTIVE OVERWRITE - FORBIDDEN

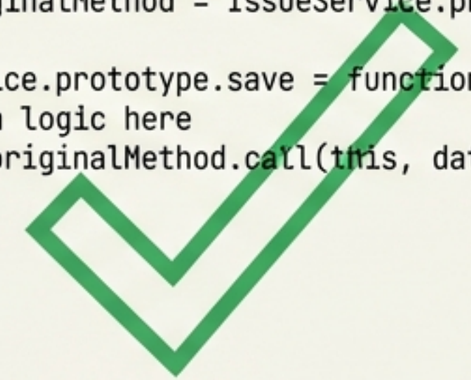
```
// COPYING ENTIRE FILE
class IssueService { ... }
```



Do not redefine classes globally.

SURGICAL PATCH - REQUIRED

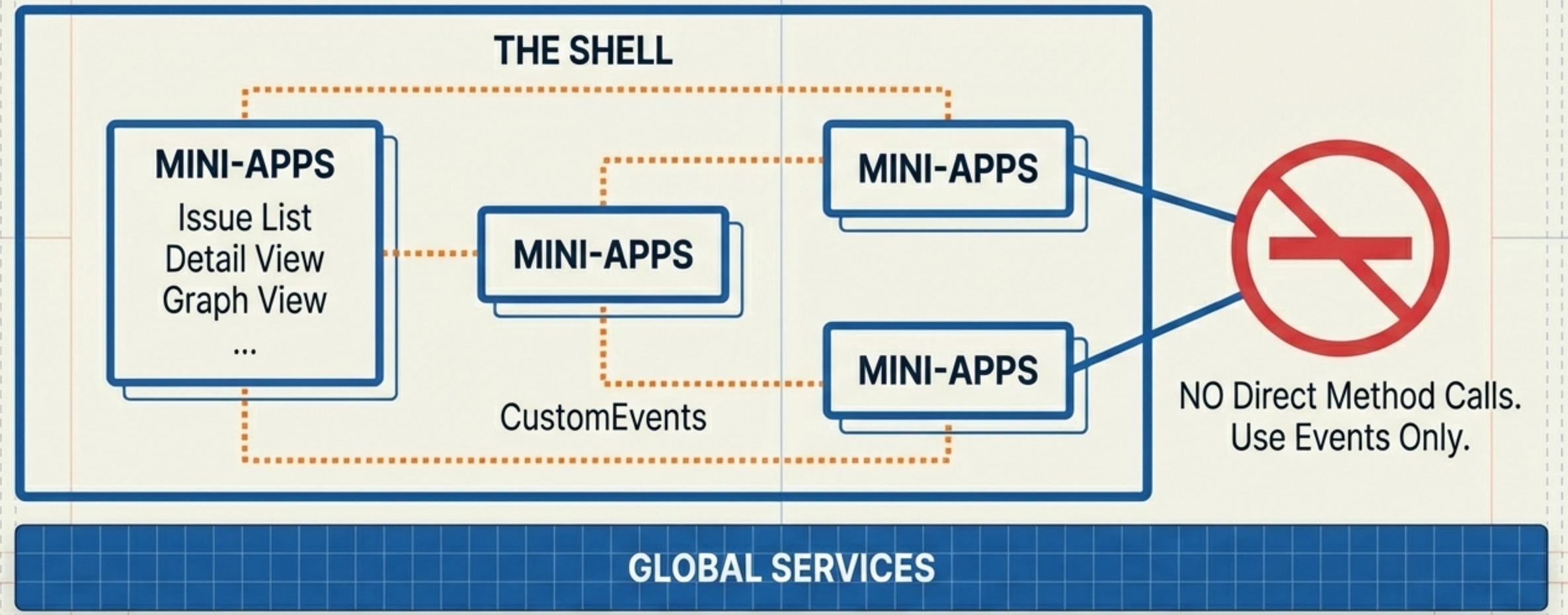
```
(function() {
  const originalMethod = IssueService.prototype.save;

  IssueService.prototype.save = function(data) {
    // Patch logic here
    return originalMethod.call(this, data);
  };
})();
```



Wrap in IIFE (Immediately Invoked Function Expression) to scope variables.

COMPONENT ARCHITECTURE & COMMUNICATION

Helvetica Now Display Blueprint Blue



Accessible by all components. Modified via surgical patching.

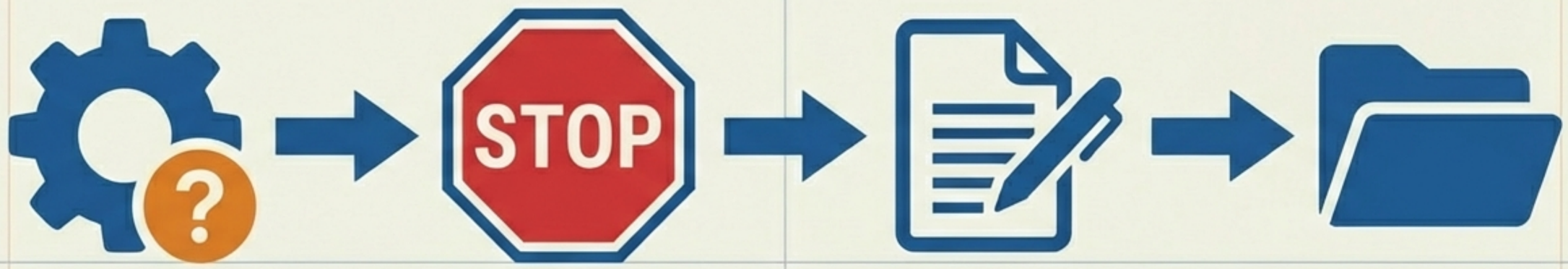
API INTEGRATION STRATEGY

METHOD	ENDPOINT	ACTION
GET	/nodes/api/nodes/{type}	List all
GET	/nodes/api/nodes/{type}/{label}	Get single
PATCH	/nodes/api/nodes/{type}/{label}	Update
POST	/nodes/api/nodes/{type}	Create
DELETE	/nodes/api/nodes/{type}/{label}	Delete
GET	/nodes/api/nodes/{type}/{label}/graph?depth=N	Get graph connections

TECHNICAL NOTE: GRAPH API RESPONSE

The 'link_type' field can return an empty string. The UI must handle this gracefully—do not assume a value exists.

PROTOCOL FOR BACKEND CHANGES



DISCOVERY
Identify missing backend feature.

HALT
Stop coding on this feature. Do not attempt Python fixes.

DOCUMENT
Draft Briefing Document.

SAVE
Save to: **docs/dev-briefs/issues-ui/**

TECHNICAL NOTE:
Filename Format: "`vX.Y.Z-ui-frontend-briefing__on__backend-changes.md`" in JetBrains Mono.

IMPLEMENTATION CHECKLIST: PHASE 1 (PREP)

MANIFEST: PHASE 1 PREP

- [ ] **SYNC:** Pull latest from `'dev'` branch.
- [ ] **CONTEXT:** Read new backend briefings & review backlog issues.
- [ ] **PLAN:** Create implementation brief in `'docs/dev-briefs/'`.
- [ ] **AUTHORIZE:** Obtain human approval before execution.

01

IMPLEMENTATION CHECKLIST: PHASE 2 (EXECUTION)

MANIFEST: PHASE 2 EXECUTION

- [ ] **STRUCTURE**: Create version folder (e.g., `v0.1.5/`).
- [ ] **DEPENDENCIES**: Create `index.html` loading dependencies in strict order.
- [ ] **METHODOLOGY**: Use surgical overrides. NO copying entire files.
- [ ] **SAFETY**: Use `IIFE` pattern for safe prototype patching.
- [ ] **ROBUSTNESS**: Handle edge cases (empty data, API errors).
- [ ] **VALIDATION**: Test with REAL API (No mocks allowed).
- [ ] **FINALISE**: Update status to ‘review’, commit, push.

02

COMMON TECHNICAL PATTERNS

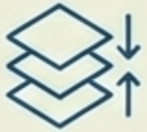
FINDING NEXT INDEX

Check `.issues/data/{type}/_index.json`.

Use `'next_index'` for new issue ID.

Increment and update file.

CSS CASCADING

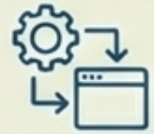


Load order is critical.

Base styles first, overrides second.

Avoid excessive `'!important'`.

EXTENDING SERVICES



Do not redefine.

Extend the prototype within an IIFE closure to preserve state.

TROUBLESHOOTING & EDGE CASES

SYMPTOM	ROOT CAUSE/FIX
• Method not found after override	Check script order in `index.html`. Base must load first.
• Issue data not saving	Verify `node_id/created_by` are exactly 8 alphanumeric chars.
• Graph shows empty labels	Handle empty '`link_type`' string in rendering logic.
• Navigation 'Back' fails	Track '`previousAppId`' in shell state.

QUICK START: IMMEDIATE ACTIONS

1. **READ CONTEXT:** Review this briefing document.
2. **VERIFY STATE:** `git branch` (Expect: `claude/implement-workbench-*`)
3. **SYNC:** `git pull origin dev`
4. **UPDATE:** `ls docs/dev-briefs/issues-ui/`
5. **TARGET:** Check backlog in `.issues/data/*/`
6. **ENGAGE:** Ask Operator 'What should I work on?'